team members up to speed not only with what the application is supposed to do, but also how it does so. For organizations that hand over projects to a different team once they go into production, the benefits of this alone can be worth the time invested in setting up the living documentation reporting.

But living documentation goes beyond describing and illustrating the features that have been built. Many teams also integrate their BDD reports with agile project management or issue-tracking systems, making it possible to focus on the state of the features planned for a particular release. Teams that do this typically rely on the living documentation to produce their release reports, if they aren't generated automatically from the BDD reports.

In the rest of this chapter, we'll look at some of these different aspects of living documentation in more detail. I'll illustrate many of the principles using Thucydides reports and a few other similar tools, but the principles aren't specific to any particular toolset.

11.2 Are we there yet? Reporting on feature readiness and feature coverage

One of the core concepts in BDD is something we'll call *feature readiness*. Features, in this sense, are just pieces of functionality that the stakeholders care about. Some teams use user stories in this role, and others prefer to distinguish user stories from higher-level features. The principle is the same in both cases: when the development team reports on progress, stakeholders are less interested in which individual tests pass or fail and are more interested in what functionality is ready to be deployed to production.

11.2.1 Feature readiness: what features are ready to deliver

In BDD terms, a feature can be considered *ready* (or *done*) when all of its acceptance criteria pass (see figure 11.3). If you can automate all of these acceptance criteria, then the automated test reports can give you a simple, concise view of the state of the features you're building. At this level, you're more interested in the overall result of all of the scenarios associated with a feature than with whether individual scenarios pass or fail.

Most BDD tools provide at least some level of reporting on feature readiness, where scenario results are aggregated at the feature level. For example, Thucydides provides feature-level reports, either directly with JBehave or with test results imported from other tools such as Cucumber, SpecFlow, and Behave. SpecFlow also provides comprehensive built-in reporting. Cucumber provides only basic feature reporting out of the box, but tools like Cucumber Reports (www.masterthought.net/section/cucumber-reporting) provide more presentable reports.

In figure 11.4, for example, you can see a feature report for a Cucumber project, generated using the Cucumber Reports library. Here the status of each feature is reported based on the overall result of the corresponding scenarios.